Algorithmen und Datenstrukturen
Übung 4 – Arrays

Aufgabe 1: Theorie

a) Welche Fehler finden sich in den folgenden Codeauszügen? Wie äußern sich die Fehler jeweils? Geben Sie **alle** Fehler in jedem Codebeispiel an.

Hinweis: Die gesuchten Fehler haben immer exakt mit den abgedruckten Zeilen zu tun. Fehler außerhalb des angegebenen Codes (z. B. fehlende Speicherfreigaben) sind nicht gesucht, ebensowenig Schönheitsmakel (z. B. schlechte Namen)!

```
1    // Beispiel A
2    int* data = malloc(5);
3    data[4] = 17
```

```
1    // Beispiel B
2    int a1[15];
3    int *a2 = a1;
4    for (int i = 0; i < sizeof(a1); ++i)
5        std::cout << a2[i];
```

```
1    // Beispiel C
2    int arr[20];
3    int *iptr = arr + 10 * sizeof(int);
4    std::cout << iptr;
5    std::cout << *iptr;
6    std::cout << arr[10 * sizeof(int)];
```

```
1    // Beispiel D
2    int[] create_sequence(int length)
3    {
4        const int sequence_length = length;
5        int sequence[sequence_length];

6        for (int i = 0; i < length; ++i)
7            *(sequence + i) = rand();        // random numbers

8        return sequence;
9    }
```

Aufgabe 2: Zeichenketten

a) Die folgende Funktion soll prüfen, ob ein C-String einen Punkt enthält:

```
// str must point to \0-terminated C string
bool has_point(const char* str)
{
    for (int i = 0; i < strlen(str); ++i)
        if (str[i] == '.') return true;

    return false;
}
```

In der Praxis stellt sich heraus, dass die obige Funktion (besonders bei deaktivierten Compiler-Optimierungen) für lange Strings recht viel Rechenzeit braucht.

- Wie kommt der hohe Rechenzeitverbrauch zustande?
- Machen Sie einen Code-Vorschlag, um die Rechenzeit zu reduzieren, wenn Sie weiterhin mit C-Strings, `strlen` und einem `int` arbeiten sollen. Behalten Sie den Suchalgorithmus bei – also kein `strchr`!
- Machen Sie einen Code-Vorschlag, der keinen `int` (und keinen anderen Index) zum Durchgehen der Zeichen benötigt. Hier sind der C-String sowie der Suchalgorithmus beizubehalten – also kein `strchr`!
- Löst es das Rechenzeitproblem, falls man statt `const char*` einen `std::string` verwendet und den Code wie nachstehend anpasst? Begründen Sie!

```
bool has_point(const std::string& str)
{
    for (int i = 0; i < str.size(); ++i)
        if (str[i] == '.') return true;

    return false;
}
```

b) Schreiben Sie eine Klasse `string` mit nachfolgenden Funktionen.

- `string::string()`

  Konstruktion eines Strings mit einer Kopie der festen Zeichenkette `""`.

- `string::string(const char* data)`

  Konstruktion aus einem `data`-Zeiger auf eine Zeichenkette. Internes Speichern einer Kopie der eingehenden Zeichenkette.

  Welche Voraussetzung muss die Zeichenkette `data` erfüllen, damit dieser Konstruktor funktioniert? Was passiert, wenn `data` ein `nullptr` ist?

- `char* string::data()`

  Abfrage der aktuell gespeicherten Zeichenkette.

  Kann diese Funktion `nullptr` zurückgeben?

- `void string::clear()`

  Löschen der gespeicherten Zeichenkette.

  Überlegen Sie in diesem Zusammenhang, was der Unterschied zwischen `nullptr`-Daten und einem leeren String `""` ist. Welche dieser beiden Varianten ist als Ergebnis der `clear()`-Methode sinnvoll?

- `void string::print()`

  Ausgeben der aktuellen Zeichenkette.

- `int string::find(char c)`

  Finden eines Zeichens `c` in der aktuellen Zeichenkette. Rückgabewert sollte der Index des Zeichens im String sein.

  Was passiert, wenn das Zeichen nicht gefunden wird?

- `void string::append(const char* data)`

  Anhängen einer weiteren Zeichenkette an den bestehenden String.

- `string::~string()`

  Destruktor. Löschen der gespeicherten Zeichenkette.

  Genügt hier ein Aufruf der Memberfunktion `clear()`?

Verwenden Sie intern nur `malloc()`, `realloc()` und `free()`, um die `string`-Zeichen zu verwalten. Nutzen Sie kein `strlen`, kein `strcpy` und kein `memcpy`. Ebenfalls nicht erlaubt ist die Verwendung von Hilfsklassen (z. B. `std::vector`, `std::string`, ...) zum internen Speichern der Zeichenkette. Für die Ausgabe darf aber die Standard-Bibliothek `iostream` verwendet werden.

Achten Sie auf eine passende Aufteilung in Header- und Quelldatei. Dokumentieren Sie Ihre Klasse. Relevant sind für den Klassennutzer interessante Spezialfälle.

Testen Sie alle Funktionen Ihrer `string`-Klasse in einer `main`-Methode. Testwürdig sind insbesondere Spezial- und Randfälle (z. B. `nullptr`-Eingaben, Suche nach dem Zeichen `\0`).

Hinweise:

- Die Fragen bei den einzelnen Punkten der Implementierung sollen Sie beim Programmieren anleiten. Sie müssen diese Fragen nicht explizit beantworten.
- Die `string`-Klasse wird in den nachfolgenden Übungsblättern noch erweitert. Es empfiehlt sich daher, den Code gut aufzubereiten und sicher abzulegen.

c) Was geht schief, wenn Sie Ihre `string`-Klasse wie folgt verwenden? Warum?

```
1    int main()
2    {
3        string s("hallo ");
4        char* data = s.data();
5        s.append("Welt!");
6        data[0] = 'H';
7        s.print();
8    }
```

Fassen Sie Ihre Beobachtungen in einen geeigneten Kommentar, um Anwender von diesem Verhalten in Kenntnis zu setzen.

d) Was passiert, wenn Sie in der `main()` einen String anlegen und in die nachfolgende Methode übergeben? Wie kommt dieses Verhalten zustande?

```
void print_string(string s) { s.print(); }
```

Hinweis: Benutzen Sie gegebenenfalls einen Debugger, um im Einzelschrittmodus den Programmablauf zu verfolgen. Es ist nicht verlangt, dass Sie Ihre `string`-Klasse so abändern, dass obiger Code korrekt läuft. Im Release-Modus maskiert die Code-Optimierung durch den Compiler eventuell das interessante Verhalten.

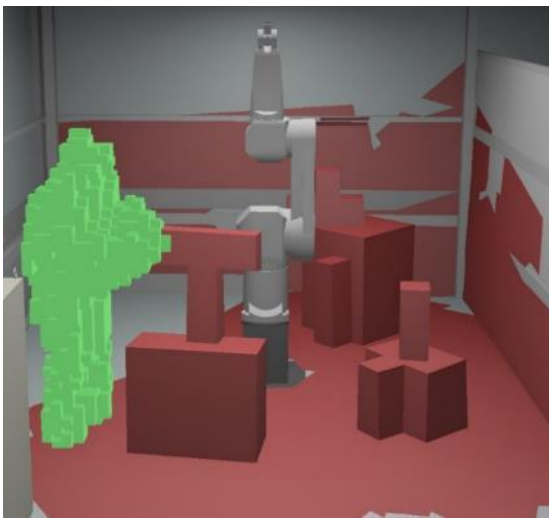
Aufgabe 3: Arrays

a) In einer Robotik-Anwendung wird die Umgebung des Roboters durch einen Sensor überwacht. Der Sensor liefert ein 3D-Array mit Voxeln zurück. In diesem 3D-Array ist für jeden diskreten Raumpunkt $(x, y, z) \in \mathbb{N}^3$ in einem Bereich von $(0, 0, 0)$ bis $(s_x - 1, s_y - 1, s_z - 1)$ als `bool` hinterlegt, ob sich an dem Ort ein Hindernis (z. B. ein Mensch) befindet oder nicht. Dabei gibt $(s_x, s_y, s_z) \in \mathbb{N}^3$ den aufspannenden Vektor des vom Sensor überwachten Raums an. Im Bild unten sehen Sie in grüner Farbe ein Beispiel für ein Hindernis in einer Voxeldarstellung.

Beantworten Sie in diesem Rahmen die folgenden Teilfragen:

- Das vom Sensor kommende 3D-Array sei im Speicher mehrstufig abzulegen. Geben Sie ein C++-Codefragment an, das für das mehrstufige Array passende Speicherblöcke allokiert und an einen Zeiger `data` auf dem Stack bindet. Sie brauchen den Speicher in dieser Aufgabe weder initialisieren, noch freigeben.
- Das vom Sensor kommende 3D-Array sei im Speicher linearisiert abgelegt. Für Algorithmen, die mit den Sensordaten arbeiten, sind zum Zugriff auf die Daten zwei Umrechnungen wichtig: Von einem Punkt $p = (p_x, p_y, p_z) \in \mathbb{N}^3$ zu einem linearen Index $i \in \mathbb{N}$ im Array sowie von einem linearen Index i zu einem Punkt (p_x, p_y, p_z). Geben Sie eine Formel zur Umrechnung von p zu i an sowie eine zugehörige Formel zur Umrechnung von i zu p.





b) Schreiben Sie eine Klasse `int_vector` für ein Array aus `int`-Zahlen.

Die Klasse soll folgende Funktionen bieten:

- Default-Konstruktor für ein leeres Array
- Copy-Konstruktor für ein bestehendes Array

- Funktion `at`, welche ein Alias für einen Eintrag an einem bestimmten Index zurückliefert
- Funktion `size`, welche die Größe des Arrays zurückgibt
- Funktion `contains`, die prüft, ob ein gegebener Integer im Array enthalten ist
- Funktion `find`, die zurückgibt, an welchem Index ein Integer im Array is
- Funktion `push_back` zum Einfügen eines `int`s am Ende des Arrays
- Funktion `pop_back` zum Entfernen eines `int`s am Ende des Arrays
- Funktion `insert` zum Einfügen eines `int`s an einem gegebenen Index, wobei die Reihenfolge der bestehenden Elemente erhalten bleibt (nicht deren Indizes, logischerweise)
- Funktion `remove` zum Löschen eines `int`s an einem gegebenen Index, wobei die Reihenfolge der verbleibenden Elemente erhalten bleibt (nicht deren Indizes, logischerweise)
- Funktion `clear`, die alle Einträge löscht
- Destruktor nach Konventionen

Dokumentieren Sie Ihren Code, wie Ihnen das angemessen erscheint. Achten Sie darauf, dass die Mischung aus Code und Kommentaren gut lesbar und verständlich ausfällt.

c) Schreiben Sie eine `main`-Methode, um Ihren `int_vector` ausgiebig zu testen. Führen Sie einige Operationen in Folge aus, so dass Sie das Ergebnisarray bereits kennen. Prüfen Sie, ob das Ergebnis von Ihrem Code ebenfalls ermittelt wird. Wiederholen Sie das für ein paar weitere Operationen bzw. Eingaben für die Funktionen.